

# Union-Find

Day 1 solution walkthrough

Competitive Programming Bootcamp

## Statement excerpts:

- ▶ *“the sets containing  $a$  and  $b$  are joined”*
- ▶ *“asking whether  $a$  and  $b$  belong to the same set”*

## Programming interpretation:

- ▶ Maintain a partition of  $N$  elements under merge operations.
- ▶ Answer each query by checking whether two elements have the same representative.
- ▶ Path compression and rank keep a million operations feasible.

**Textbook connection:** CP4 2.4.2 Union-Find Disjoint Sets

**Core technique:** Disjoint set union

- ▶ The problem maintains dynamic connectivity among numbered elements.
- ▶ Operation '=' joins two sets.
- ▶ Operation '?' asks whether two elements currently share a set.

- ▶ The graph only gains connections; it never deletes them.
- ▶ Connectivity queries after unions are exactly the union-find use case.
- ▶ Path compression and union by rank keep operations effectively constant.

- ▶ Initialize  $\text{parent}[i] = i$ ,  $\text{rank}[i] = 0$ , and  $\text{size}[i] = 1$ .
- ▶ `findSet` walks parent pointers to the representative and compresses the path.
- ▶ `isSameSet` compares the two representatives.
- ▶ `unionSet` attaches the lower-rank root under the higher-rank root.
- ▶ Process commands and buffer yes/no answers.

- ▶ Representatives name the current connected component.
- ▶ Path compression rewires visited nodes directly to the root.
- ▶ Union by rank prevents long chains from forming.

- ▶ Initially every element is alone, matching the problem state.
- ▶ Each union merges exactly the two components containing its endpoints.
- ▶ Two elements are connected exactly when their representatives are equal.

**Complexity:**  $O((N + Q) \alpha(N))$  time and  $O(N)$  memory;  $\alpha(N)$  is effectively constant.

- ▶ The iterative find avoids Python recursion-limit problems.
- ▶ Buffer output because many query answers may be printed.
- ▶ The extra size and set-count fields are useful library features, even if not all are needed here.



Source lines: unionfind.py:52-82

```
def __init__(self, numItems: int, numSets: int = None) -> None:
    if numItems < 1:
        raise ValueError("numItems needs to be greater or equal to 1")
    elif numSets is not None and numItems < numSets:
        raise ValueError("numSets must not be greater than numItems")

    self._numSet = numItems
    self._parents = list(range(numItems))
    self._ranks = [0] * numItems
    self._sizes = [1] * numItems
    if numSets is None:
        numSets = numItems
    else:
        while self._numSet > numSets:
            self.unionSet(random.randrange(numItems), random.randrange(
                numItems))
```

## What this chunk does

- ▶ Each element begins as the parent of itself.
- ▶ rank supports shallow merges; size supports optional set-size queries.
- ▶ `_numSet` tracks how many disjoint components remain.

Source lines: unionfind.py:112-143

```
def findSet(self, element: int) -> int:
    children = []
    toFind = element

    while toFind != self._parents[toFind]:
        children.append(toFind)
        toFind = self._parents[toFind]

    for c in children:
        self._parents[c] = toFind

    return toFind
```

## What this chunk does

- ▶ The loop walks parent pointers until it reaches the root representative.
- ▶ Visited children are stored so their parent can be rewritten directly to the root.
- ▶ Future find operations on those nodes become much faster.

Source lines: unionfind.py:145-155

```
def isSameSet(self, element1: int, element2: int) -> bool:  
    return self.findSet(element1) == self.findSet(element2)
```

## What this chunk does

- ▶ Two elements are in the same set exactly when their representatives match.
- ▶ Calling findSet also compresses paths along the way.

Source lines: unionfind.py:157-182

```
def unionSet(self, element1: int, element2: int) -> None:
    s1 = self.findSet(element1)
    s2 = self.findSet(element2)

    if s1 == s2:
        return None

    if self._ranks[s1] > self._ranks[s2]:
        s1, s2 = s2, s1
    elif self._ranks[s1] == self._ranks[s2]:
        self._ranks[s2] += 1

    self._parents[s1] = s2

    self._numSet -= 1
    self._sizes[s2] += self._sizes[s1]
```

## What this chunk does

- ▶ The method first finds both roots, because only roots should be linked.
- ▶ The smaller-rank root is attached below the larger-rank root.
- ▶ When ranks tie, one rank increases to reflect the possible height growth.

Source lines: unionfind.py:210-230

```
lines = sys.stdin.read().strip().splitlines()
n = int(lines[0].split()[0])
output = []

ufds = UnionFind(n)

for command in lines[1:]:
    line = command.split()
    op = line[0]
    e1 = int(line[1])
    e2 = int(line[2])

    match op:
        case "?":
            output.append("yes\n" if ufds.isSameSet(e1, e2) else "no\n")
        case "=":
            ufds.unionSet(e1, e2)

sys.stdout.write("".join(output))
```

## What this chunk does

- ▶ The input is read once, then commands are interpreted line by line.
- ▶ A question command appends yes or no based on isSameSet.
- ▶ An equals command merges the two sets with unionSet.

**Solution file:** `practice_contests/day_1/unionfind.py`

**Problem:** <https://open.kattis.com/problems/unionfind>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.